

OWASP TOP 10

Assignment: OWASP Top 10 Vulnerabilities

Name: Nelson Ngumo

Course: AfricaHackon Academy — Cohort 6

Date: 16 September 2026

1. Broken Access Control

Description: This occurs when restrictions on what authenticated users are allowed to do aren't properly enforced, letting attackers act outside their intended permissions — for example viewing, editing, or deleting other users' data by simply changing an ID in a URL, or accessing admin-only functions as a regular user.

Mitigation: Enforce access control checks on every request server-side (never rely on client-side checks or hidden UI elements), deny by default, and use a single, centrally-tested authorization mechanism rather than scattering ad-hoc checks throughout the codebase.

2. Cryptographic Failures

Description: Sensitive data (passwords, card numbers, personal data, session tokens) is exposed because it's transmitted or stored without adequate encryption, or with weak/outdated algorithms — previously known as 'Sensitive Data Exposure,' this category focuses on the root cause rather than the symptom.

Mitigation: Encrypt sensitive data at rest and in transit using strong, up-to-date algorithms (e.g. TLS 1.2+, AES-256), avoid storing sensitive data you don't need, and use proper key management rather than hard-coded or reused keys.

3. Injection

Description: Untrusted user input is passed to an interpreter (SQL, OS command, LDAP, etc.) as part of a command or query without proper validation, allowing an attacker to alter its meaning — classic examples are SQL injection and OS command injection.

Mitigation: Use parameterized queries / prepared statements and ORM libraries instead of building queries via string concatenation, validate and sanitize all user input, and apply the principle of least privilege to database accounts.

4. Insecure Design

Description: A category focused on flaws in the design and architecture of an application itself — risks baked in before a single line of code is written — such as missing threat modeling, insufficient business-logic validation, or trusting client-side data by design, which no amount of clean implementation can fully fix.

Mitigation: Adopt secure-by-design practices from the start: threat modeling during design, secure design patterns and reference architectures, and defining and testing explicit trust boundaries and abuse-case scenarios rather than only 'happy path' requirements.

5. Security Misconfiguration

Description: Insecure default configurations, incomplete or ad-hoc setups, open cloud storage, unnecessary features enabled, verbose error messages leaking stack traces, or missing security headers — essentially, the application or its environment isn't hardened.

Mitigation: Establish a repeatable, automated hardening process (and re-apply it across dev/staging/prod), disable unused features/ports/services/accounts, keep software patched, and review configurations regularly rather than only at initial deployment.

6. Vulnerable and Outdated Components

Description: Using libraries, frameworks, or other software components with known vulnerabilities, or components that are unsupported/out of date, without tracking their versions or the vulnerabilities disclosed against them — this includes both direct and transitive dependencies.

Mitigation: Maintain an inventory of all components and their versions (an SBOM), continuously monitor for CVEs affecting them (e.g. via Dependabot, OWASP Dependency-Check, or Snyk), and patch/upgrade promptly rather than deferring updates indefinitely.

7. Identification and Authentication Failures

Description: Weaknesses in how an application confirms a user's identity, authenticates them, or manages sessions — such as allowing weak/default passwords, missing multi-factor authentication, exposing session IDs in URLs, or not invalidating sessions properly on logout.

Mitigation: Enforce strong password policies and multi-factor authentication, use secure, well-tested session management (random, sufficiently long session IDs, rotated on login, invalidated on logout/timeout), and rate-limit or lock out repeated failed login attempts.

8. Software and Data Integrity Failures

Description: Code and infrastructure that don't protect against integrity violations — for example, relying on plugins, libraries, or CI/CD pipelines from untrusted sources without verifying their integrity, or auto-updating software without validating the update's signature, opening the door to supply-chain attacks.

Mitigation: Verify the integrity of software/data using digital signatures or checksums, ensure CI/CD pipelines have proper access control and configuration review, and avoid pulling dependencies from untrusted repositories or over unencrypted channels.

9. Security Logging and Monitoring Failures

Description: Insufficient logging, monitoring, or alerting means breaches and suspicious activity go undetected for long periods — without an audit trail, an organization can't detect, investigate, or respond to an active attack in time to limit damage.

Mitigation: Log all security-relevant events (logins, access-control failures, input validation failures) with enough context to investigate, centralize logs, and set up real-time alerting/monitoring so anomalies trigger a timely response rather than being found in a post-mortem.

10. Server-Side Request Forgery (SSRF)

Description: Occurs when an application fetches a remote resource (e.g. a URL) without validating the user-supplied URL, letting an attacker force the server to make requests to unintended destinations — including internal-only systems, cloud metadata endpoints, or internal services behind a firewall.

Mitigation: Validate and sanitize/allow-list any user-supplied URLs or hostnames before the server fetches them, deny requests to internal/private IP ranges and cloud metadata endpoints by default, and isolate the fetching functionality on a segmented network with no unnecessary internal access.